

■ Stock Market Analyzer

Application Python d'analyse de données financières en temps réel

Auteur	Souheil
Type de projet	Application Python avec interface graphique
Domaine	Data Engineering / Analyse financière
Langage	Python 3.10
Bibliothèques	Tkinter, yfinance, pandas, matplotlib, openpyxl
Source de données	Yahoo Finance (API yfinance)
Marché ciblé	Top 10 actions françaises (CAC 40)

Sommaire

1. Présentation du projet
2. Objectifs
3. Fonctionnalités
4. Architecture du projet
5. Technologies utilisées
6. Pipeline ETL
7. Captures d'écran de l'application
8. Code source (extraits)
9. Cas d'usage
10. Améliorations futures
11. Installation et lancement
12. Conclusion

1. Présentation du projet

Le **Stock Market Analyzer** est une application Python qui permet d'analyser des données financières en temps réel à partir du marché boursier. L'application offre une interface graphique simple permettant de sélectionner plusieurs actions, d'extraire leurs données financières, de les comparer et de générer des visualisations ainsi que des exports Excel.

Ce projet simule un outil d'analyse financière proche de ceux utilisés dans les domaines de la banque, de la finance et de la data analyse. Il met en œuvre un pipeline complet de type **ETL** (Extract – Transform – Load) intégrant une interface utilisateur, une couche de récupération de données via API, une transformation structurée des données et plusieurs canaux de restitution (tableau, graphique, export Excel).

2. Objectifs du projet

L'objectif principal est de démontrer la capacité à :

- Automatiser la récupération de données financières via une API publique
- Structurer et transformer des données brutes en informations exploitables
- Comparer plusieurs actions simultanément sur des indicateurs clés
- Visualiser graphiquement les performances des marchés
- Générer des rapports exploitables au format Excel
- Concevoir une interface graphique fonctionnelle avec Tkinter

3. Fonctionnalités principales

3.1 Sélection multiple d'actions

L'utilisateur peut sélectionner une ou plusieurs entreprises parmi le top 10 du marché français (LVMH, TotalEnergies, BNP Paribas, Airbus, Sanofi, L'Oréal, Schneider Electric, AXA, Danone, Safran).

3.2 Récupération des données financières

Pour chaque action sélectionnée, le système récupère :

- Prix actuel
- Prix d'ouverture
- Plus haut et plus bas de la journée
- Volume échangé
- Capitalisation boursière
- Horodatage de récupération

3.3 Analyse comparative et visualisation

Les données sont regroupées dans un **DataFrame pandas** afin de comparer les performances des différentes entreprises. Un graphique en barres généré avec matplotlib permet de visualiser instantanément l'écart de prix entre les actions sélectionnées.

3.4 Export Excel

Les résultats peuvent être exportés au format **.xlsx** via `openpyxl` pour une utilisation externe (reporting, analyse approfondie, archivage).

4. Architecture du projet

```
Stock-Market-Analyzer/  
■  
■■■ app.py           # Application principale (Tkinter)  
■■■ requirements.txt  # Dépendances Python  
■■■ README.md        # Documentation du projet  
■■■ data/            # Données exportées (Excel)  
■■■ graphs/          # Graphiques générés  
■■■ screenshots/     # Captures d'écran de l'application
```

5. Technologies utilisées

Composant	Technologie	Rôle
Langage	Python 3.10	Langage principal
Interface	Tkinter	GUI native multi-plateforme
API financière	yfinance	Récupération données Yahoo Finance
Manipulation	pandas	Structuration en DataFrame
Visualisation	matplotlib	Génération de graphiques
Export	openpyxl	Écriture de fichiers Excel

6. Pipeline ETL

Le projet suit une logique standard **Extract – Transform – Load** :

6.1 Extract

Les données sont récupérées depuis l'API Yahoo Finance via la bibliothèque **yfinance**. Pour chaque symbole boursier (ex. MC.PA pour LVMH), l'objet `Ticker` expose les informations de marché en temps réel.

6.2 Transform

Les données brutes (dictionnaire JSON) sont nettoyées, filtrées et structurées dans un **DataFrame pandas**. Seuls les champs pertinents (prix, ouverture, haut/bas, volume, capitalisation) sont conservés et horodatés.

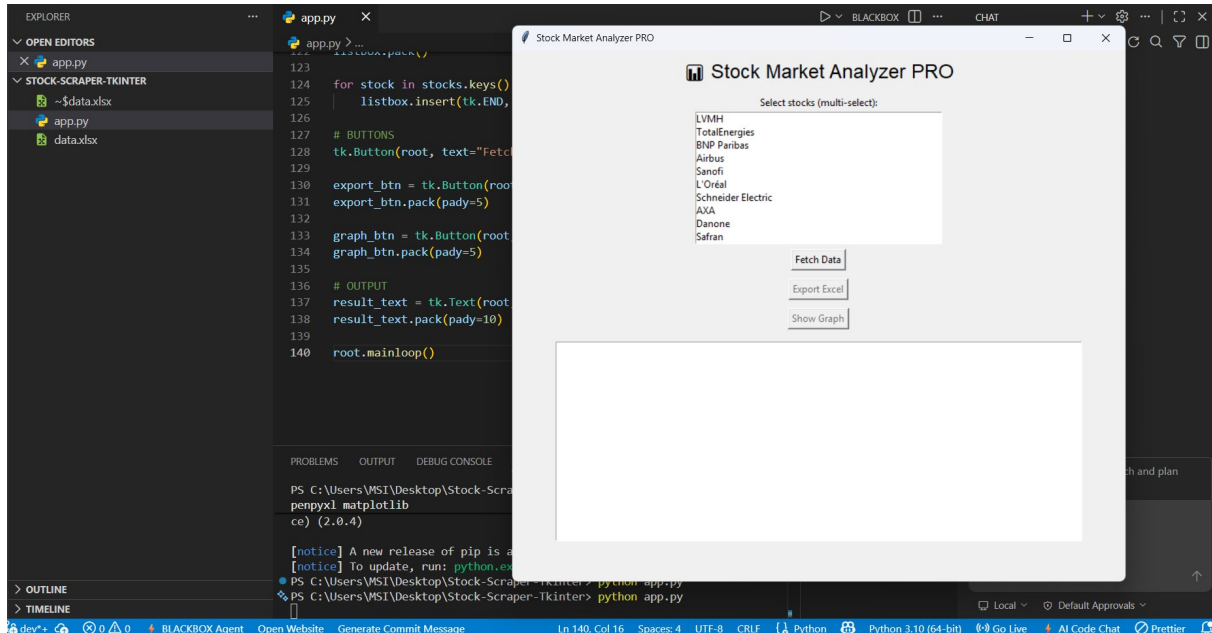
6.3 Load

Les données sont chargées dans trois canaux de sortie : (1) l'interface graphique sous forme tabulaire, (2) un fichier Excel via `openpyxl`, et (3) un graphique `matplotlib` pour la visualisation comparative.

7. Captures d'écran de l'application

7.1 Interface principale au lancement

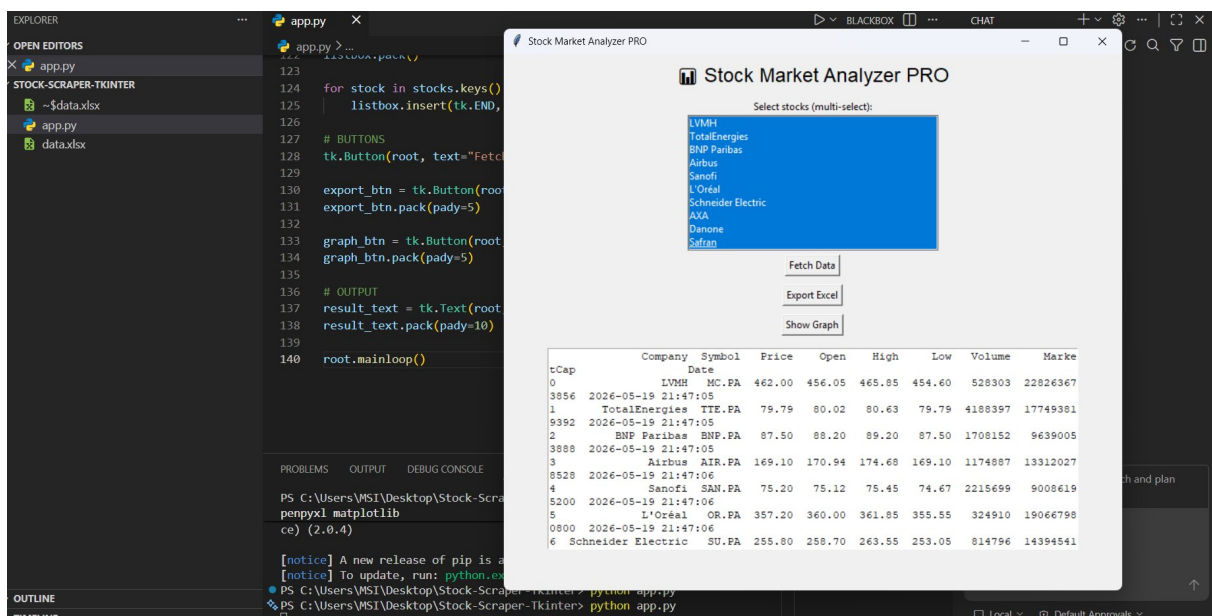
L'application affiche la liste des 10 actions françaises sélectionnables. Les boutons *Export Excel* et *Show Graph* sont désactivés tant qu'aucune donnée n'a été récupérée.



Capture 1 — Interface au démarrage

7.2 Affichage des données récupérées

Après sélection des actions et clic sur *Fetch Data*, les données sont récupérées via yfinance et affichées dans la zone de texte. Les boutons d'export et de visualisation deviennent actifs.



Capture 2 — Données récupérées (10 actions)

7.3 Visualisation graphique

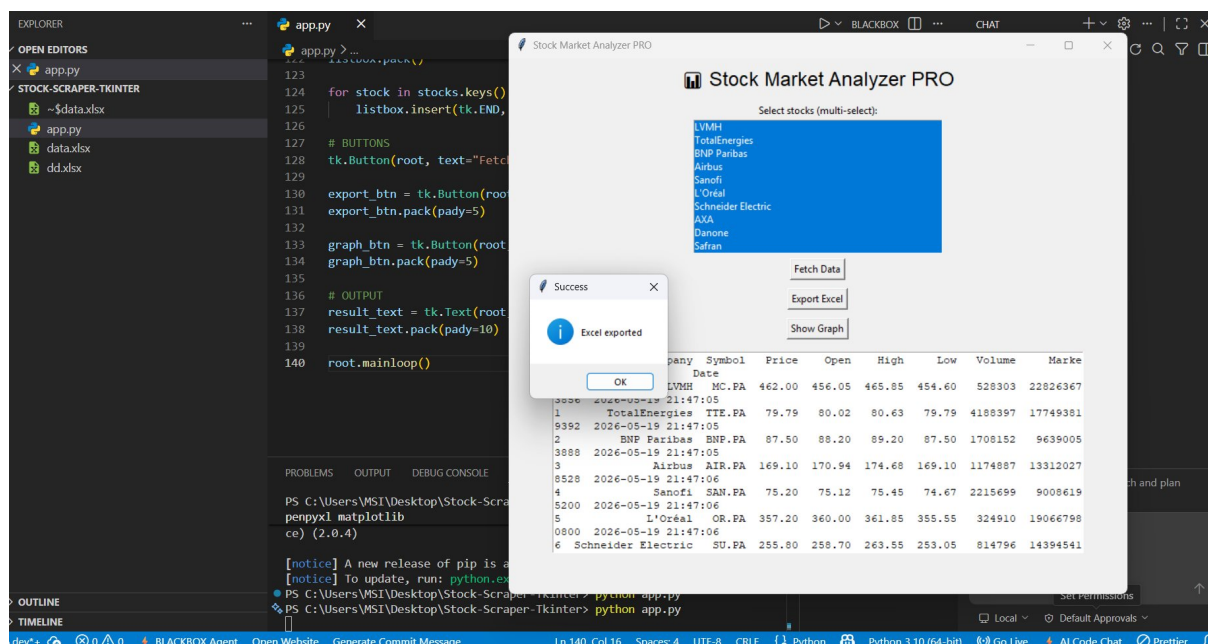
Le bouton *Show Graph* ouvre une fenêtre matplotlib comparant les prix des actions sélectionnées sous forme de diagramme en barres.



Capture 3 — Graphique comparatif des prix

7.4 Confirmation d'export Excel

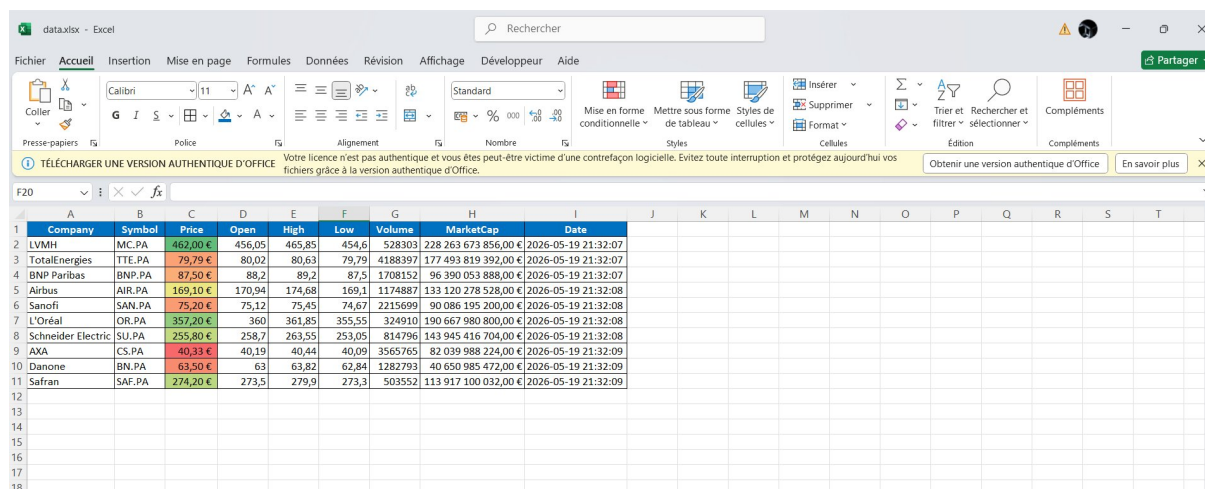
Une notification confirme l'export réussi du fichier .xlsx.



Capture 4 — Confirmation d'export Excel

7.5 Fichier Excel exporté

Le fichier généré contient un formatage conditionnel automatique sur la colonne *Price* ainsi qu'un en-tête mis en forme.



The screenshot shows an Excel spreadsheet with the following data:

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
1	Company	Symbol	Price	Open	High	Low	Volume	MarketCap	Date											
2	LVMH	MC.PA	462,00 €	456,05	465,85	454,6	528303	228 263 673 856,00 €	2026-05-19 21:32:07											
3	TotalEnergies	TTE.PA	79,79 €	80,02	80,63	79,79	4188397	177 493 819 392,00 €	2026-05-19 21:32:07											
4	BNP Paribas	BNP.PA	87,50 €	88,2	89,2	87,5	1708152	96 390 053 888,00 €	2026-05-19 21:32:07											
5	Airbus	AIR.PA	169,10 €	170,94	174,68	169,1	1174887	133 120 278 528,00 €	2026-05-19 21:32:08											
6	Sanofi	SAN.PA	75,20 €	75,12	75,45	74,67	2215699	90 086 195 200,00 €	2026-05-19 21:32:08											
7	L'Oréal	OR.PA	357,20 €	360	361,85	355,55	324910	190 667 980 800,00 €	2026-05-19 21:32:08											
8	Schneider Electric	SU.PA	255,80 €	258,7	263,55	253,05	814796	143 945 416 704,00 €	2026-05-19 21:32:08											
9	AXA	CS.PA	40,33 €	40,19	40,44	40,09	3565765	82 039 988 224,00 €	2026-05-19 21:32:09											
10	Danone	BN.PA	63,50 €	63	63,82	62,84	1282793	40 650 985 472,00 €	2026-05-19 21:32:09											
11	Safran	SAF.PA	274,20 €	273,5	279,9	273,3	503552	113 917 100 032,00 €	2026-05-19 21:32:09											
12																				
13																				
14																				
15																				
16																				
17																				
18																				

Capture 5 — Données exportées dans Excel

8. Code source (extraits)

8.1 Définition des actions cibles

```
stocks = {
    "LVMH": "MC.PA",
    "TotalEnergies": "TTE.PA",
    "BNP Paribas": "BNP.PA",
    "Airbus": "AIR.PA",
    "Sanofi": "SAN.PA",
    "L'Oréal": "OR.PA",
    "Schneider Electric": "SU.PA",
    "AXA": "CS.PA",
    "Danone": "BN.PA",
    "Safran": "SAF.PA",
}
```

8.2 Fonction de récupération des données

```
def fetch_all_data(selected_keys):
    all_data = []
    for name in selected_keys:
        symbol = stocks[name]
        stock = yf.Ticker(symbol)
        info = stock.info
        if "regularMarketPrice" in info:
            all_data.append({
                "Company": name,
                "Symbol": symbol,
                "Price": info.get("regularMarketPrice"),
                "Open": info.get("open"),
                "High": info.get("dayHigh"),
                "Low": info.get("dayLow"),
                "Volume": info.get("volume"),
                "MarketCap": info.get("marketCap"),
                "Date": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
            })
    return pd.DataFrame(all_data)
```

8.3 Visualisation graphique

```
def show_graph():
    df = root.df
    plt.figure(figsize=(10, 6))
    plt.bar(df["Company"], df["Price"])
    plt.title("Stock Price Comparison")
    plt.xlabel("Companies")
    plt.ylabel("Price (€)")
    plt.xticks(rotation=45)
    plt.tight_layout()
    plt.show()
```

9. Cas d'usage

- Analyse financière simplifiée pour un investisseur particulier
- Suivi rapide d'un portefeuille d'actions françaises
- Apprentissage du traitement de données financières en Python
- Démonstration de compétences en Python et data engineering
- Base pour un dashboard financier plus avancé (Streamlit, Dash)

10. Améliorations futures

Analyse temporelle — Suivi de l'évolution historique des prix sur plusieurs mois/années.

Graphiques avancés — Chandeliers japonais (candlestick), courbes lissées, comparaisons.

Indicateurs techniques — Moyennes mobiles, RSI, MACD, bandes de Bollinger.

Persistance — Connexion à une base SQLite/PostgreSQL pour l'historisation.

Dashboard web — Migration vers une interface Streamlit ou Dash.

Machine Learning — Modèle de prédiction des prix (régression, LSTM).

11. Installation et lancement

Installation des dépendances :

```
pip install yfinance pandas matplotlib openpyxl
```

Lancement de l'application :

```
python app.py
```

12. Conclusion

Le projet **Stock Market Analyzer** illustre la mise en œuvre complète d'un pipeline ETL en Python : récupération automatisée de données financières via une API publique, transformation structurée à l'aide de pandas, et restitution multi-canal (interface graphique, graphique matplotlib, export Excel).

Au-delà du résultat fonctionnel, ce projet démontre la capacité à concevoir une application end-to-end couvrant l'ensemble de la chaîne de la donnée — de l'extraction à la visualisation — tout en proposant une expérience utilisateur simple et intuitive. Il constitue une base solide pour des évolutions plus avancées (analyse temporelle, indicateurs techniques, modèles prédictifs) et reflète des compétences transférables aux métiers de la data engineering, de la BI et de l'analyse financière.